

Binary Search

Binary Search is an Efficient Searching Technique used to find a particular Element in a Sorted Array.

It Follows the Divide-and-Conquer Approach, Instead of checking Every Element one by one.

It Compares the Target with the Middle Element and Eliminates half of the Search space at Each step.

a) How Binary Search operates on Sorted Data.

Consider the Sorted Array.

$$A = [8, 12, 16, 20, 24, 32]$$

(i) Find the Middle Element

$$\text{middle} = 16$$

(ii) Compare 20 with 16

Since $20 > 16$, Search only the right half

(iii) Right half is $[20, 24, 32]$

civ) Middle Element is 24

Since $20 < 24$ search the Left half

cv) The remaining element is 20

* Therefore 20 is found

Binary Search ($A[n]$, Key)

low = 0

high = $n-1$

while low $\leq$ high:

mid = $(\text{low} + \text{high}) / 2$

if $A[\text{mid}] == \text{Key}$:

return "Element Found".

else if $\text{Key} < A[\text{mid}]$

high = mid - 1

else:

low = mid + 1

return "Element Not Found"

b) performance when order is Maintained versus Disturbed

when order is Maintained.

when the elements remain Sorted, Binary Search performs very Efficiently

For Every Comparison, Approximately half of the remaining elements are eliminated.

For Example:

1000 $\rightarrow$ 500 $\rightarrow$ 250 $\rightarrow$ 125 $\rightarrow$ 62 $\rightarrow$...

Therefore, the Time Complexity is: $O(\log n)$

For $n=1000$

$\log_2(1000) \approx 10$ Comparisons

Thus the Binary Search can be search a Large

Datasets using very Few Comparisons

$\rightarrow$ Suppose the Sorted Array:

[8, 12, 16, 20, 24, 32]

becomes [8, 20, 12, 32, 16, 24]

vi) useful in computer systems

vii) predictable performance

Time Complexities

Situation	Complexity
-----------	------------

Best Case	$O(1)$
-----------	--------

Average Case	$O(\log n)$
--------------	-------------

Worst Case	$O(\log n)$
------------	-------------

$$\text{Sorting} + \text{Searching} = O(n \log n) + O(\log n) = O(n \log n)$$

Binary Search is an Efficient Searching Algorithm that works by repeatedly Dividing a Sorted Dataset into Two Halves.

Now the Elements are not in sorted order.

If Binary Search is directly Applied, it may eliminate the wrong half of the Array. Consequently, it can report that an element is Absent Even when the element is Actually present.

∴ Binary Search Cannot Guarantee Correct results on unsorted Data

Overall: $O(n \log n) + O(\log n) = O(n \log n)$

c) Importance of Maintaining Sorted Data

Maintaining Sorted Data is very Important when Frequent Searching is required.

Advantages:

- i) Fast Searching
- ii) Efficient for Large Datasets
- iii) Reduces Comparisons
- iv) Usefull for Databases
- v) useful for Dictionaries